UNIVERSITAS NEGERI YOGYAKARTA
FAKULTAS TEKNIK
PROGRAM STUDI TEKNIK INDUSTRI

LAPORAN PRAKTIKUM

APLIKASI WEB DAN MOBILE

Semester Genap 2025/2026

MODUL 8 [Asynchronous JavaScript (Fetch API, JSON, REST API Sederhana)]

Nama Mahasiswa	:	Vanyra Raraswara Eka Putri
NIM	:	23051430013
Kelas	:	A
Tanggal Praktikum	:	17 April 2026
Dosen Pengampu	:	Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT.

Dikumpulkan paling lambat 7 hari setelah pelaksanaan praktikum

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul Asynchronous JavaScript (Fetch API, JSON, REST API Sederhana) [Pertemuan 8]
Topik Utama	Asynchronous JavaScript (Fetch API, JSON, REST API Sederhana)
Sub-Topik	Asynchronous JavaScript (Fetch API, JSON, REST API Sederhana)
Durasi Praktikum	340 Menit
Tanggal Praktikum	17 April 2026
Nama Mahasiswa	Vanyra Raraswara Eka Putri
NIM	23051430013
Kelas	A

B. TUJUAN PEMBELAJARAN

1	Memahami perbedaan kode sinkronus dan asinkronus.
2	Mampu menggunakan `fetch()` untuk mengambil data dari server eksternal (API).
3	Memahami format data JSON (JavaScript Object Notation).
4	Menerapkan penanganan error saat request API gagal.

C. ALAT DAN BAHAN

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor

2	Google Chrome / Browser Modern	Versi terbaru	Testing & Debugging
3	Node.js & npm	Node 18+ / npm 9+	Runtime JS (Modul tertentu)
4	Aplikasi Web & Modul Manual Praktikum	Versi Terbaru	Panduan Praktikum

D. DASAR TEORI

Asynchronous JavaScript merupakan paradigma pemrograman yang memungkinkan eksekusi kode berjalan tanpa memblokir proses lainnya saat menunggu operasi yang memakan waktu (seperti permintaan jaringan) selesai. Teknik ini menjadi fondasi utama dalam pengembangan aplikasi web modern karena mencegah antarmuka pengguna "membeku" saat berkomunikasi dengan server. **Fetch API** adalah antarmuka JavaScript modern yang menggantikan XMLHttpRequest (XHR) untuk melakukan HTTP request secara asinkronus. Fetch API bekerja dengan mengembalikan objek **Promise** yang mewakili penyelesaian (atau kegagalan) suatu operasi asinkronus, memungkinkan pengembang untuk menangani respons melalui metode .then() dan error melalui .catch() dengan struktur kode yang lebih bersih dan mudah dibaca.

JSON (JavaScript Object Notation) adalah format pertukaran data ringan yang menjadi standar de facto dalam komunikasi API modern. JSON menggunakan sintaks yang mirip dengan objek literal JavaScript sehingga mudah di-parse menggunakan method response.json() yang juga mengembalikan Promise. Konsep **REST API** memungkinkan aplikasi web mengakses sumber daya dari server melalui endpoint URL standar, dengan method HTTP seperti GET (mengambil data), POST (mengirim data), PUT (memperbarui data), dan DELETE (menghapus data). Dalam praktikum ini, JSONPlaceholder digunakan sebagai simulasi REST API publik yang menyediakan data dummy untuk keperluan pembelajaran dan prototipe cepat.

Promise dan Error Handling merupakan mekanisme inti dalam mengelola operasi asinkronus di JavaScript. Sebuah Promise memiliki tiga status: *pending* (sedang berproses), *fulfilled* (berhasil), dan *rejected* (gagal). Chaining method .then() memungkinkan eksekusi berurutan dari operasi asinkronus, sementara .catch() berfungsi menangkap dan menangani error yang terjadi di sepanjang rantai Promise. Pendekatan modern menggunakan **Async/Await** yang diperkenalkan dalam ES2017 (ES8) memungkinkan penulisan kode asinkronus dengan gaya sinkronus, di mana kata kunci await menghentikan eksekusi fungsi (tanpa memblokir thread utama) hingga Promise diselesaikan. Penanganan error pada async/await dilakukan menggunakan blok try...catch yang lebih intuitif. Kedua pendekatan ini penting untuk memastikan aplikasi web tetap responsif dan mampu memberikan umpan balik yang tepat kepada pengguna saat terjadi kegagalan komunikasi dengan server.

D.1 Konsep Utama

Konsep utama pada Pertemuan 8 mencakup pemahaman mendasar tentang perbedaan antara eksekusi kode sinkronus dan asinkronus dalam JavaScript. Eksekusi sinkronus memproses baris kode secara berurutan dan dapat menyebabkan antarmuka pengguna "membeku" saat menunggu operasi lambat seperti permintaan jaringan, sementara eksekusi asinkronus memungkinkan kode lain terus berjalan tanpa menunggu. Fetch API menjadi tools utama untuk melakukan HTTP request asinkronus dengan mengembalikan objek Promise yang dapat ditangani melalui chaining .then() dan .catch(). Data yang diterima dari server umumnya berbentuk JSON string, yang kemudian di-parse menjadi objek JavaScript menggunakan response.json() agar dapat dimanipulasi dan ditampilkan ke antarmuka pengguna. Penanganan error menjadi aspek krusial untuk memberikan pengalaman pengguna yang baik saat terjadi kegagalan jaringan, respons server yang tidak valid, atau kesalahan parsing data. Pendekatan modern menggunakan sintaks Async/Await menyederhanakan penulisan kode asinkronus sehingga lebih mudah dibaca dan di-debug, dengan penanganan error melalui blok try...catch.

D.2 Keterkaitan dengan Teknik Industri

Implementasi asynchronous JavaScript dan Fetch API memiliki relevansi signifikan dalam konteks Teknik Industri, khususnya pada pengembangan sistem informasi manufaktur berbasis web. Penelitian Dasmito (2019) dalam skripsinya di Universitas Hasanuddin mendemonstrasikan penggunaan teknologi web modern termasuk Fetch API untuk membangun sistem pemeliharaan mesin pada Industri Kecil Menengah (IKM). Studi tersebut menunjukkan bahwa aplikasi web dengan service worker (yang memanfaatkan fetch event) mampu meningkatkan performa akses data secara signifikan pada kondisi jaringan terbatas, suatu skenario yang umum ditemui di lingkungan industri yang berlokasi di daerah dengan infrastruktur internet belum memadai.

Lebih lanjut, konsep asynchronous data fetching menjadi tulang punggung dashboard monitoring real-time yang banyak diimplementasikan di lantai produksi. Data dari sensor mesin, sistem MES (Manufacturing Execution System), atau database inventori dapat diambil secara periodik melalui Fetch API tanpa mengganggu aktivitas operator yang sedang menggunakan aplikasi. Kemampuan untuk mengirim data (POST) juga memungkinkan pengembangan fitur pelaporan insiden, input hasil produksi, atau pencatatan downtime mesin secara langsung dari antarmuka web ke server pusat. Integrasi ini mendukung prinsip Industri 4.0 di mana konektivitas dan pertukaran data real-time menjadi kunci efisiensi operasional dan pengambilan keputusan berbasis data di sektor manufaktur.

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

E.1 Langkah 1 – Mengenal JSON Placeholder

Langkah pertama yang dilakukan adalah dengan menggunakan layanan gratis `https://jsonplaceholder.typicode.com/users` sebagai simulasi database "Karyawan" atau "Agen Supplier".

Kode Hasil HTML:

Screenshot Hasil:

Gambar E.1 – Screenshot Hasil Langkah 1

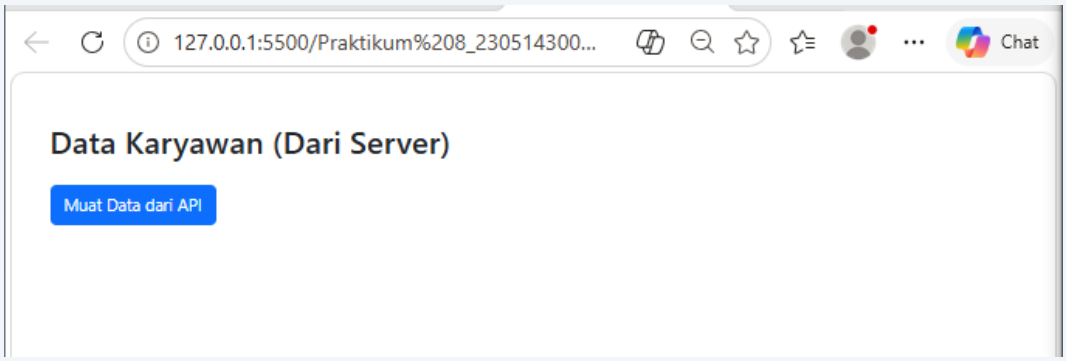
E.2 Langkah 2 – Membuat HTML untuk Menampilkan Data

Selanjutnya, membuat file `api_demo.html` yang digunakan untuk menampilkan data.

Kode Hasil HTML:

```
<!DOCTYPE html>
<html lang="id">
<head>
<meta charset="UTF-8">
<title>Integrasi API Data Karyawan</title>
<link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body class="container mt-5">
<h2 class="mb-4">Data Karyawan (Dari Server)</h2>
<div class="mb-3">
<button id="btnLoad" class="btn btn-primary">Muat Data dari API</button>
<div id="loading" class="spinner-border text-primary d-none"
role="status"></div>
</div>
<div class="row" id="containerKaryawan">
<!-- Card karyawan akan muncul di sini -->
</div>
<script src="api_script.js"></script>
</body>
</html>
```

Screenshot Hasil:



Gambar E.2 – Screenshot hasil Langkah 2

E.3 Langkah 3 – Fetch Data dengan .then() (Promise)

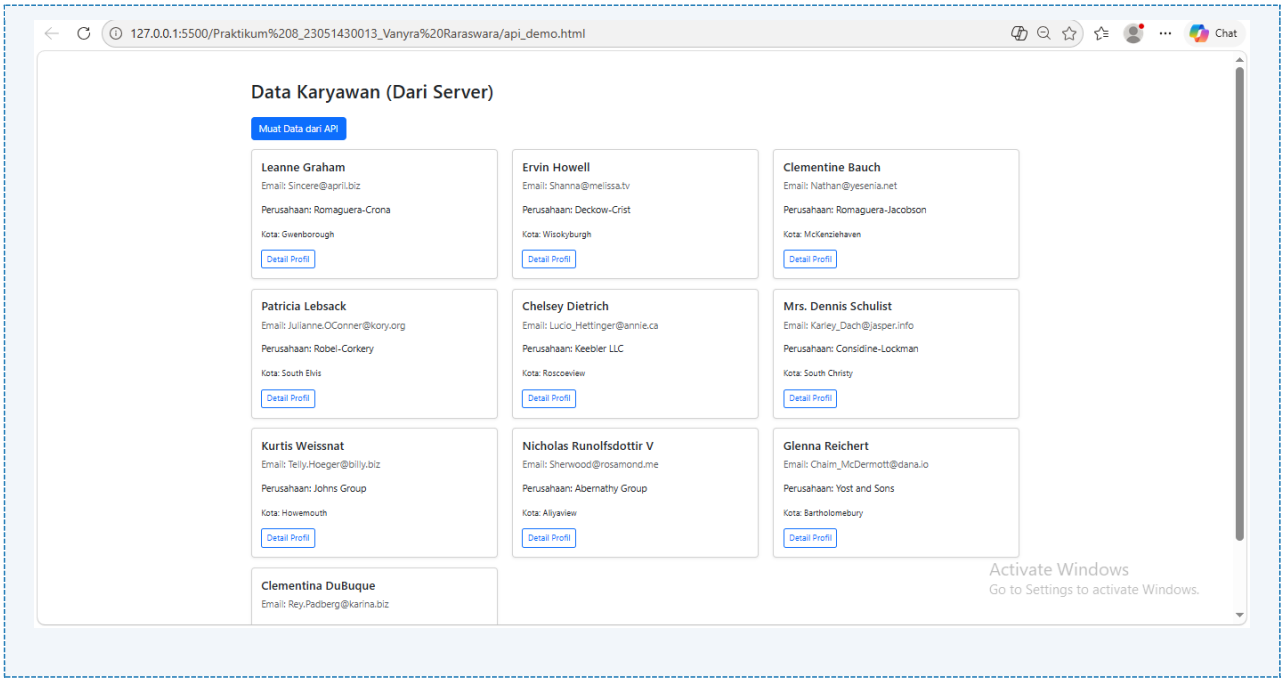
Selanjutnya, membuat file `api_script.js`

Kode Hasil JavaScript:

```
const btnLoad = document.getElementById('btnLoad');
const container = document.getElementById('containerKaryawan');
const loading = document.getElementById('loading');

const API_URL = 'https://jsonplaceholder.typicode.com/users';

btnLoad.addEventListener('click', function() {
    loading.classList.remove('d-none');
    container.innerHTML = '';

    fetch(API_URL)
        .then(function(response) {
            if (!response.ok) {
                throw new Error('Gagal mengambil data');
            }
            return response.json();
        })
        .then(function(dataKaryawan) {
            console.log(dataKaryawan);
            renderData(dataKaryawan);
        })
        .catch(function(error) {
            container.innerHTML = `<div class="alert alert-danger">Error:
${error.message}</div>`;
```

```
        })
        .finally(function() {
            loading.classList.add('d-none');
        });
});
function renderData(data) {
    data.forEach(function(karyawan) {
        const col = document.createElement('div');
        col.className = 'col-md-4 mb-3';
        col.innerHTML = `
            <div class="card h-100 shadow-sm">
                <div class="card-body">
                    <h5 class="card-title">${karyawan.name}</h5>
                    <p class="card-text text-muted">Email:
${karyawan.email}</p>
                    <p class="card-text">Perusahaan:
${karyawan.company.name}</p>
                    <p class="card-text"><small>Kota:
${karyawan.address.city}</small></p>
                    <a href="#" class="btn btn-sm btn-outline-primary">Detail
Profil</a>
                </div>
            </div>
        `;
        container.appendChild(col);
    });
}
```

Screenshot Hasil:



Gambar E.3 – Screenshot hasil Langkah 3

E.4 Langkah 4 – Fetch dengan Async/Await (Modern Style)

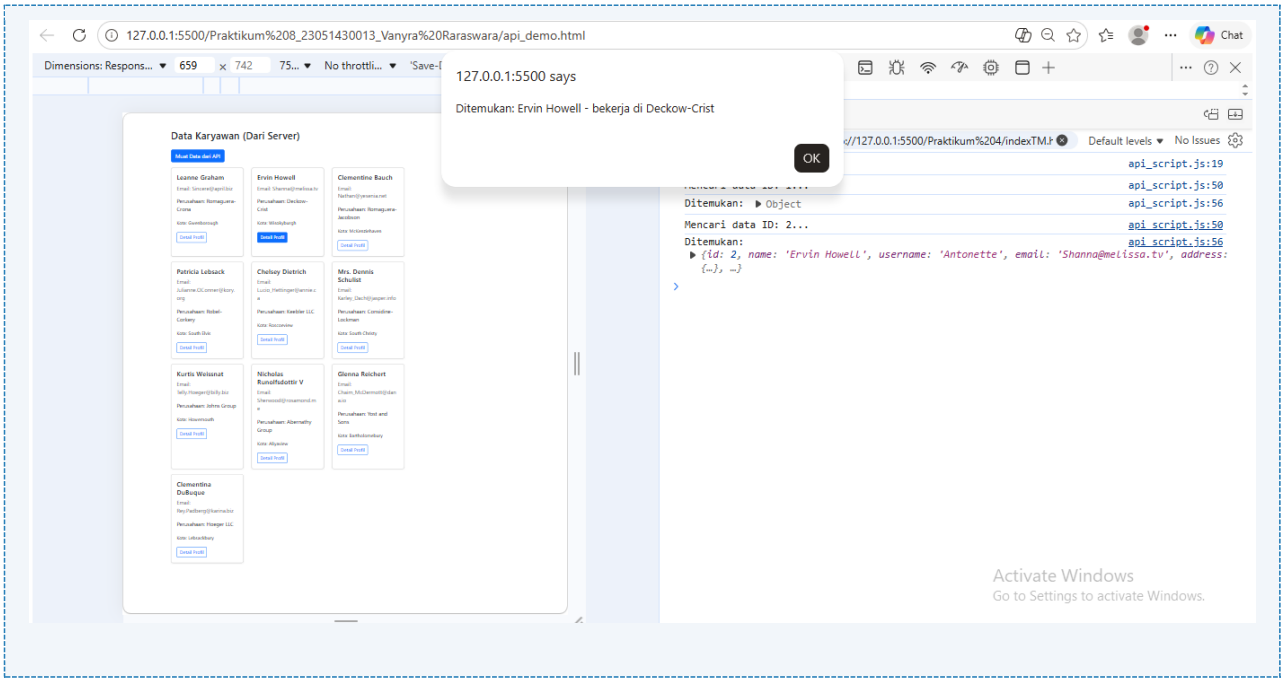
Selanjutnya, Async/Await membuat kode asinkron terlihat seperti sinkron untuk lebih mudah dibaca. Disini membuat fungsi baru untuk mencari karyawan berdasarkan ID.

Kode Hasil JavaScript:

```
const btnLoad = document.getElementById('btnLoad');
const container = document.getElementById('containerKaryawan');
const loading = document.getElementById('loading');

const API_URL = 'https://jsonplaceholder.typicode.com/users';

btnLoad.addEventListener('click', function() {
    loading.classList.remove('d-none');
    container.innerHTML = '';

    fetch(API_URL)
        .then(function(response) {
            if (!response.ok) {
                throw new Error('Gagal mengambil data');
            }
            return response.json();
        })
        .then(function(dataKaryawan) {
            console.log(dataKaryawan);
            renderData(dataKaryawan);
        })
        .catch(function(error) {
            container.innerHTML = `<div class="alert alert-danger">Error:
${error.message}</div>`;
        })
        .finally(function() {
            loading.classList.add('d-none');
        });
});
function renderData(data) {
    data.forEach(function(karyawan) {
        const col = document.createElement('div');
        col.className = 'col-md-4 mb-3';
        col.innerHTML = `
            <div class="card h-100 shadow-sm">
                <div class="card-body">
                    <h5 class="card-title">${karyawan.name}</h5>
                    <p class="card-text text-muted">Email:
${karyawan.email}</p>
                    <p class="card-text">Perusahaan:
${karyawan.company.name}</p>
                    <p class="card-text"><small>Kota:
${karyawan.address.city}</small></p>
                    <a href="javascript:void(0)" class="btn btn-sm btn-
outline-primary" onclick="cariKaryawan(${karyawan.id})">Detail Profil</a>
                </div>
```

```
            </div>
        `;
        container.appendChild(col);
    });
}

async function cariKaryawan(id) {
    try {
        console.log(`Mencari data ID: ${id}...`);
        const response = await
fetch(`https://jsonplaceholder.typicode.com/users/${id}`);
        if (!response.ok) {
            throw new Error('Data tidak ditemukan');
        }
        const data = await response.json();
        console.log("Ditemukan:", data);
        alert(`Ditemukan: ${data.name} - bekerja di ${data.company.name}`);
    } catch (error) {
        console.error(error);
        alert(error.message);
    }
}
```

Screenshot Hasil:



Gambar E.3 – Screenshot hasil Langkah 4

F. LATIHAN DAN TUGAS MANDIRI

F.1 Latihan 1 – Posting Data

Pertanyaan / Instruksi Latihan:

> Buat tombol "Tambah Karyawan Baru". Gunakan method `fetch` dengan opsi `{ method: 'POST', body: ... }` untuk mengirim data dummy ke API. Cek response di console.

Jawaban / Kode Solusi HTML:

```
<!DOCTYPE html>
<html lang="id">
<head>
<meta charset="UTF-8">
<title>Integrasi API Data Karyawan</title>
<link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body class="container mt-5">
<h2 class="mb-4">Data Karyawan (Dari Server)</h2>
<div class="mb-3">
    <button id="btnLoad" class="btn btn-primary">Muat Data dari API</button>
    <button id="btnTambah" class="btn btn-success ms-2">+ Tambah
Karyawan</button>
```

```
    <div id="loading" class="spinner-border text-primary d-none"
role="status"></div>
</div>
<div class="row" id="containerKaryawan">
<!-- Card karyawan akan muncul di sini -->
</div>
<script src="api1_script.js"></script>
</body>
</html>
```

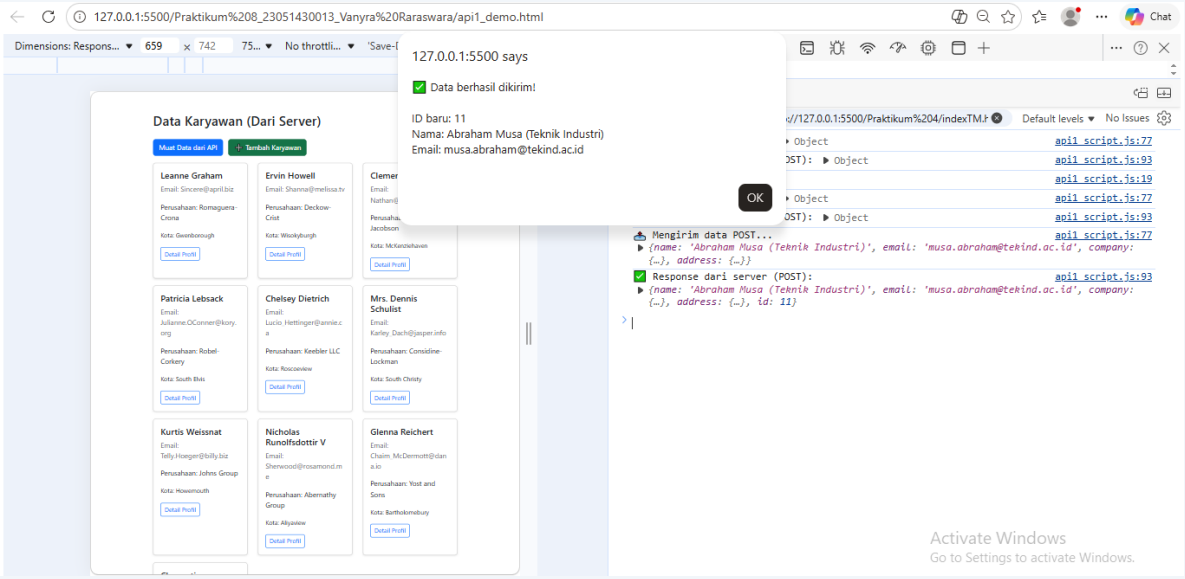
Jawaban / Kode Solusi JavaScript:

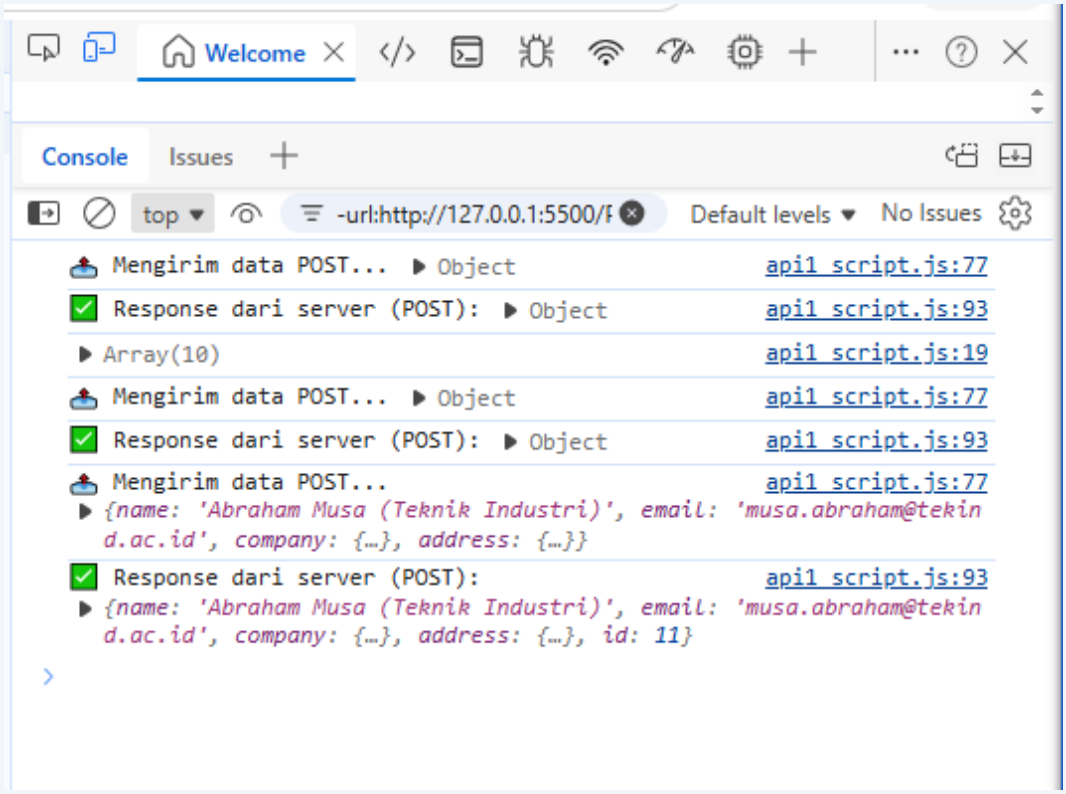
```
const btnLoad = document.getElementById('btnLoad');
const container = document.getElementById('containerKaryawan');
const loading = document.getElementById('loading');

const API_URL = 'https://jsonplaceholder.typicode.com/users';

btnLoad.addEventListener('click', function() {
    loading.classList.remove('d-none');
    container.innerHTML = '';

    fetch(API_URL)
        .then(function(response) {
            if (!response.ok) {
                throw new Error('Gagal mengambil data');
            }
            return response.json();
        })
        .then(function(dataKaryawan) {
            console.log(dataKaryawan);
            renderData(dataKaryawan);
        })
        .catch(function(error) {
            container.innerHTML = `<div class="alert alert-danger">Error:
${error.message}</div>`;
        })
        .finally(function() {
            loading.classList.add('d-none');
        });
});
function renderData(data) {
    data.forEach(function(karyawan) {
        const col = document.createElement('div');
```

```
        col.className = 'col-md-4 mb-3';
        col.innerHTML = `
            <div class="card h-100 shadow-sm">
                <div class="card-body">
                    <h5 class="card-title">${karyawan.name}</h5>
                    <p class="card-text text-muted">Email:
${karyawan.email}</p>
                    <p class="card-text">Perusahaan:
${karyawan.company.name}</p>
                    <p class="card-text"><small>Kota:
${karyawan.address.city}</small></p>
                    <a href="javascript:void(0)" class="btn btn-sm btn-
outline-primary" onclick="cariKaryawan(${karyawan.id})">Detail Profil</a>
                </div>
            </div>
        `;
        container.appendChild(col);
    });
}

async function cariKaryawan(id) {
    try {
        console.log(`Mencari data ID: ${id}...`);
        const response = await
fetch(`https://jsonplaceholder.typicode.com/users/${id}`);
        if (!response.ok) {
            throw new Error('Data tidak ditemukan');
        }
        const data = await response.json();
        console.log("Ditemukan:", data);
        alert(`Ditemukan: ${data.name} - bekerja di ${data.company.name}`);
    } catch (error) {
        console.error(error);
        alert(error.message);
    }
}
const btnTambah = document.getElementById('btnTambah');

btnTambah.addEventListener('click', function() {
    const dataBaru = {
        name: 'Abraham Musa (Teknik Industri)',
        email: 'musa.abraham@tekind.ac.id',
        company: {
            name: 'PT. Manufaktur Maju'
        },
        address: {
```

```
            city: 'Bandung'
        }
    };

    console.log('📤 Mengirim data POST...', dataBaru);

    fetch('https://jsonplaceholder.typicode.com/users', {
        method: 'POST',
        headers: {
            'Content-Type': 'application/json'
        },
        body: JSON.stringify(dataBaru)
    })
        .then(function(response) {
            if (!response.ok) {
                throw new Error('Gagal menambah data');
            }
            return response.json();
        })
        .then(function(data) {
            console.log('✅ Response dari server (POST):', data);
            alert('✅ Data berhasil dikirim!\n\n' +
                  'ID baru: ' + data.id + '\n' +
                  'Nama: ' + data.name + '\n' +
                  'Email: ' + data.email);
        })
        .catch(function(error) {
            console.error('❌ Error:', error);
            alert('Gagal menambah data: ' + error.message);
        });
});
```

Screenshot Hasil:





Gambar F.1 Screenshot Hasil Latihan 1

F.2 Latihan 2 – Filter Array

Pertanyaan / Instruksi Latihan:

> Sebelum data dirender ke layar dengan `forEach`, lakukan filter terlebih dahulu. Tampilkan HANYA karyawan yang tinggal di kota yang mengandung huruf "s" (misal: "Southside", "London"). Petunjuk: Gunakan `data.filter(...)` sebelum `data.forEach(...)`.

Jawaban / Kode Solusi HTML:

```html
<!DOCTYPE html>
<html lang="id">
<head>
<meta charset="UTF-8">
<title>Integrasi API Data Karyawan</title>
<link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body class="container mt-5">
<h2 class="mb-4">Data Karyawan (Dari Server)</h2>
<div class="mb-3">
    <button id="btnLoad" class="btn btn-primary">Muat Data dari API</button>
    <button id="btnTambah" class="btn btn-success ms-2">Tambah
Karyawan</button>
    <div id="loading" class="spinner-border text-primary d-none"
role="status"></div>
</div>
<!-- LATIHAN 2: Input Filter Array -->
<div class="mb-3">
    <label for="filterKota" class="form-label">🔍 Filter berdasarkan
Kota:</label>
    <input type="text" id="filterKota" class="form-control"
placeholder="Contoh: Gwenborough, Wisokyburgh, South Elvis..." style="max-
width: 350px;">
    <small class="text-muted">Ketik nama kota, lalu klik "Muat Data" untuk
melihat hasil filter</small>
</div>
<div class="row" id="containerKaryawan">
<!-- Card karyawan akan muncul di sini -->
</div>
<script src="api2_script.js"></script>
</body>
</html>
```

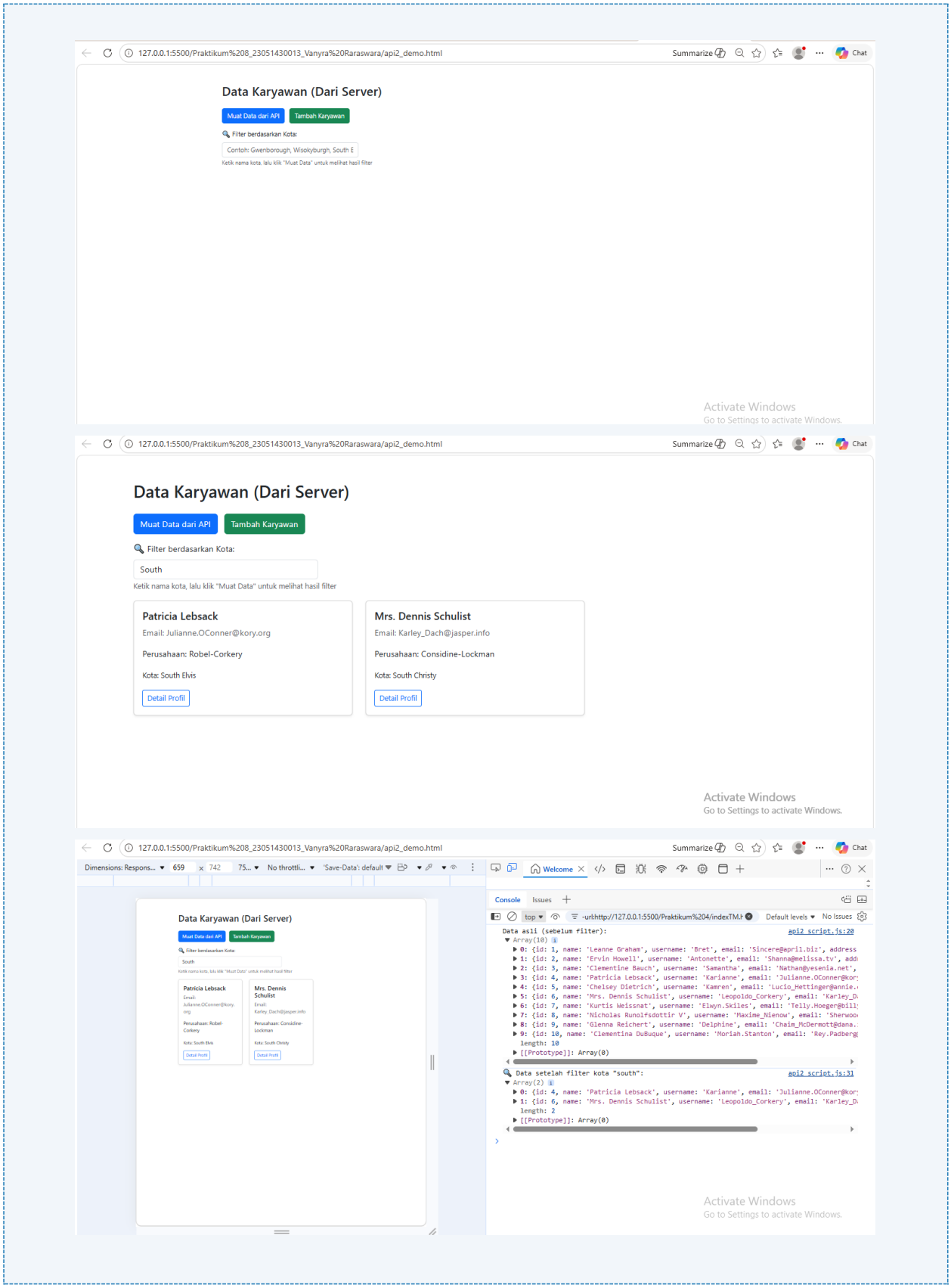
Jawaban / Kode Solusi JavaScript:

```
const btnLoad = document.getElementById('btnLoad');
const container = document.getElementById('containerKaryawan');
const loading = document.getElementById('loading');

const API_URL = 'https://jsonplaceholder.typicode.com/users';

btnLoad.addEventListener('click', function() {
    loading.classList.remove('d-none');
    container.innerHTML = '';

    fetch(API_URL)
        .then(function(response) {
            if (!response.ok) {
                throw new Error('Gagal mengambil data');
            }
            return response.json();
        })
        .then(function(dataKaryawan) {
    // Data berhasil didapat
    console.log('Data asli (sebelum filter):', dataKaryawan);

    // ===== LATIHAN 2: FILTER ARRAY BERDASARKAN KOTA =====
    const filterKota =
document.getElementById('filterKota').value.trim().toLowerCase();

    let dataTersaring = dataKaryawan;

    if (filterKota !== '') {
        dataTersaring = dataKaryawan.filter(function(karyawan) {
            return karyawan.address.city.toLowerCase().includes(filterKota);
        });
        console.log(`🔍 Data setelah filter kota "${filterKota}":`,
dataTersaring);
    }

    renderData(dataTersaring);
})
        .catch(function(error) {
            container.innerHTML = `<div class="alert alert-danger">Error:
${error.message}</div>`;
        })
        .finally(function() {
            loading.classList.add('d-none');
        });
```

```
});
function renderData(data) {
    data.forEach(function(karyawan) {
        const col = document.createElement('div');
        col.className = 'col-md-4 mb-3';
        col.innerHTML = `
            <div class="card h-100 shadow-sm">
                <div class="card-body">
                    <h5 class="card-title">${karyawan.name}</h5>
                    <p class="card-text text-muted">Email:
${karyawan.email}</p>
                    <p class="card-text">Perusahaan:
${karyawan.company.name}</p>
                    <p class="card-text"><small>Kota:
${karyawan.address.city}</small></p>
                    <a href="javascript:void(0)" class="btn btn-sm btn-
outline-primary" onclick="cariKaryawan(${karyawan.id})">Detail Profil</a>
                </div>
            </div>
        `;
        container.appendChild(col);
    });
}

async function cariKaryawan(id) {
    try {
        console.log(`Mencari data ID: ${id}...`);
        const response = await
fetch(`https://jsonplaceholder.typicode.com/users/${id}`);
        if (!response.ok) {
            throw new Error('Data tidak ditemukan');
        }
        const data = await response.json();
        console.log("Ditemukan:", data);
        alert(`Ditemukan: ${data.name} - bekerja di ${data.company.name}`);
    } catch (error) {
        console.error(error);
        alert(error.message);
    }
}
const btnTambah = document.getElementById('btnTambah');

btnTambah.addEventListener('click', function() {
    const dataBaru = {
        name: 'Abraham Musa (Teknik Industri)',
        email: 'musa.abraham@tekind.ac.id',
```

```
        company: {
            name: 'PT. Manufaktur Maju'
        },
        address: {
            city: 'Bandung'
        }
    };

    console.log('📤 Mengirim data POST...', dataBaru);

    fetch('https://jsonplaceholder.typicode.com/users', {
        method: 'POST',
        headers: {
            'Content-Type': 'application/json'
        },
        body: JSON.stringify(dataBaru)
    })
        .then(function(response) {
            if (!response.ok) {
                throw new Error('Gagal menambah data');
            }
            return response.json();
        })
        .then(function(data) {
            console.log('✅ Response dari server (POST):', data);
            alert('✅ Data berhasil dikirim!\n\n' +
                  'ID baru: ' + data.id + '\n' +
                  'Nama: ' + data.name + '\n' +
                  'Email: ' + data.email);
        })
        .catch(function(error) {
            console.error('❌ Error:', error);
            alert('Gagal menambah data: ' + error.message);
        });
});
```

Screenshot Hasil:



Gambar F.2 Screenshot Hasil Latihan 2

F.3 Tugas Proyek Mini – Integrasi Data Produksi

Deskripsi proyek mini:

> Anggap API `https://jsonplaceholder.typicode.com/posts` adalah database "Laporan Insiden".
> a. Tampilkan daftar 10 laporan insiden terbaru (Title = Judul Insiden, Body = Deskripsi).
> b. Di setiap card, tambahkan tombol "Tindak Lanjut". Saat diklik, `alert()` menampilkan "Tiket ID sedang diproses oleh Tim Maintenance".

Penjelasan pendekatan/solusi yang digunakan:

Penyelesaian ini menggunakan pendekatan Asynchronous JavaScript dengan memanfaatkan Fetch API untuk mengambil data dari JSONPlaceholder. Endpoint /posts digunakan sebagai representasi “database laporan insiden” pabrik. Data yang diterima dalam format JSON kemudian diproses menggunakan method .slice(0, 10) untuk membatasi tampilan menjadi 10 laporan terbaru. Setiap data insiden dirender secara dinamis ke dalam card Bootstrap menggunakan DOM Manipulation (createElement dan innerHTML). Untuk memenuhi kebutuhan interaksi (tugas b), ditambahkan tombol “Tindak Lanjut” pada setiap card yang memanggil fungsi tindakLanjut() menggunakan atribut onclick. Fungsi tersebut menampilkan alert yang berisi ID tiket dan pesan konfirmasi bahwa tiket sedang diproses oleh Tim Maintenance. Pendekatan ini memungkinkan aplikasi menampilkan data eksternal secara real-time tanpa perlu backend sendiri, sekaligus memberikan simulasi alur kerja penanganan insiden di lingkungan industri.

Salinan Kode Utama HTML:

```html
<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Laporan Insiden - Maintenance</title>
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body class="container mt-5">
```

```
    <h2 class="mb-4">🚨 Laporan Insiden Pabrik (10 Terbaru)</h2>

    <div class="mb-3">
        <button id="btnLoadInsiden" class="btn btn-danger">📋 Muat Laporan
Insiden</button>
        <div id="loadingInsiden" class="spinner-border text-danger d-none"
role="status"></div>
    </div>

    <div class="row" id="containerInsiden">
        <!-- Card insiden akan muncul di sini -->
    </div>

    <script src="insidenTM_script.js"></script>
</body>
</html>
```

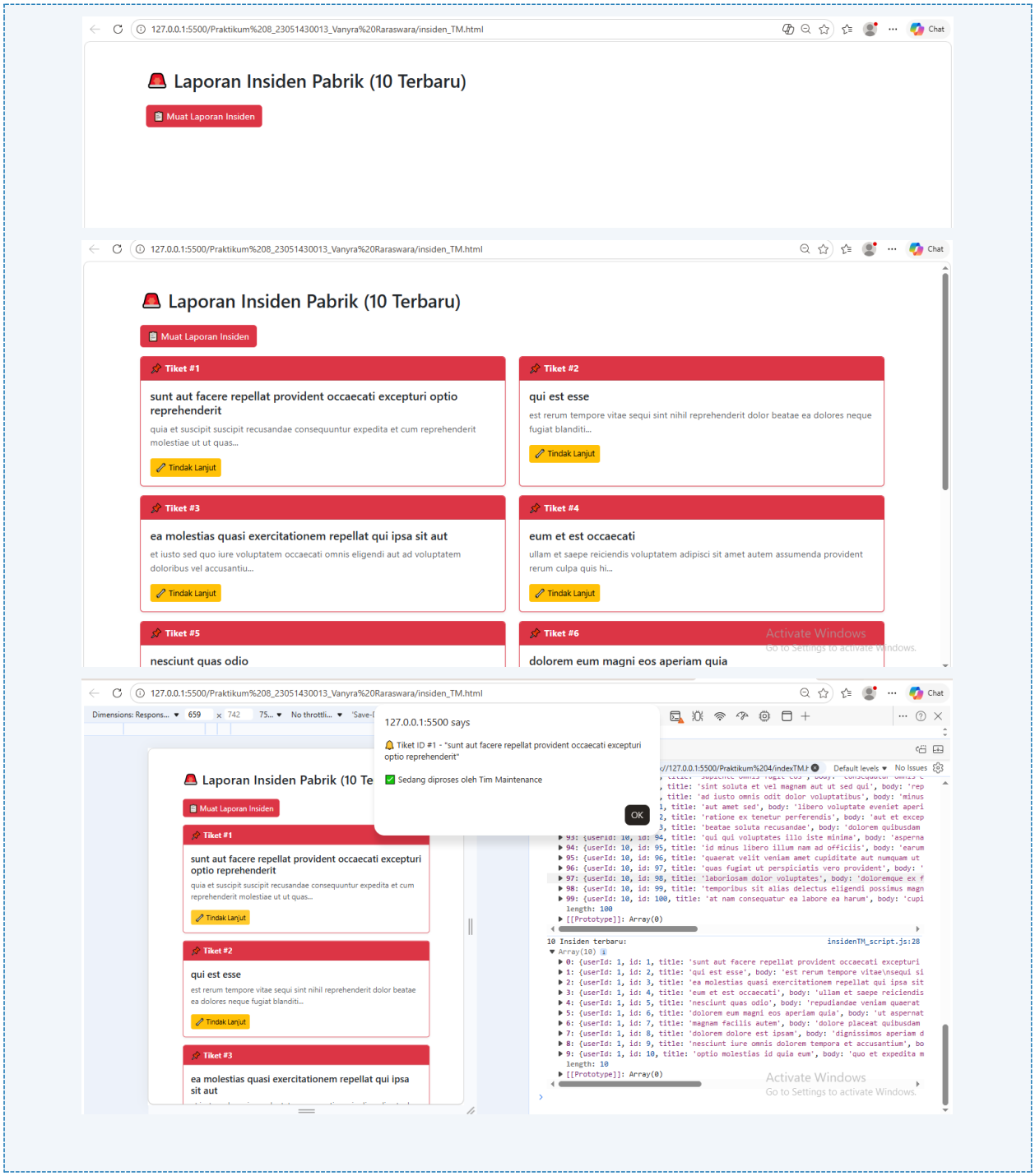
Salinan Kode Utama JavaScript:

```
// ===== SELEKSI ELEMEN =====
const btnLoad = document.getElementById('btnLoadInsiden');
const container = document.getElementById('containerInsiden');
const loading = document.getElementById('loadingInsiden');

// Endpoint API untuk Posts (sebagai simulasi Laporan Insiden)
const API_URL = 'https://jsonplaceholder.typicode.com/posts';

// ===== EVENT LISTENER & FETCH =====
btnLoad.addEventListener('click', function() {
    // Tampilkan loading
    loading.classList.remove('d-none');
    container.innerHTML = ''; // Bersihkan konten lama

    // Fetch Data
    fetch(API_URL)
        .then(function(response) {
            if (!response.ok) {
                throw new Error('Gagal mengambil data insiden');
            }
            return response.json();
        })
        .then(function(dataInsiden) {
            console.log('Data insiden (asli):', dataInsiden);

            // Ambil hanya 10 data pertama
            const sepuluhInsiden = dataInsiden.slice(0, 10);
```

```
            console.log('10 Insiden terbaru:', sepuluhInsiden);

            renderInsiden(sepuluhInsiden);
        })
        .catch(function(error) {
            console.error('Error:', error);
            container.innerHTML = `<div class="alert alert-danger">❌ Error:
${error.message}</div>`;
        })
        .finally(function() {
            loading.classList.add('d-none');
        });
});

// ===== FUNGSI RENDER INSIDEN =====
function renderInsiden(data) {
    data.forEach(function(insiden) {
        const col = document.createElement('div');
        col.className = 'col-md-6 mb-3';

        // Potong body/deskripsi agar tidak terlalu panjang
        const deskripsiSingkat = insiden.body.length > 100
            ? insiden.body.substring(0, 100) + '...'
            : insiden.body;

        col.innerHTML = `
            <div class="card h-100 shadow-sm border-danger">
                <div class="card-header bg-danger text-white">
                    <strong>📌 Tiket #${insiden.id}</strong>
                </div>
                <div class="card-body">
                    <h5 class="card-title">${insiden.title}</h5>
                    <p class="card-text text-muted">${deskripsiSingkat}</p>
                    <button class="btn btn-sm btn-warning"
onclick="tindakLanjut(${insiden.id}, '${insiden.title.replace(/'/g,
"\\'")}')">
                        🔧 Tindak Lanjut
                    </button>
                </div>
            </div>
        `;
        container.appendChild(col);
    });
}

// ===== FUNGSI TINDAK LANJUT (Tugas b) =====
```

```
function tindakLanjut(id, judul) {
    alert(`🔔 Tiket ID #${id} - "${judul}"\n\n✅ Sedang diproses oleh Tim
Maintenance`);
}

// Agar fungsi bisa dipanggil dari onclick di HTML
window.tindakLanjut = tindakLanjut;
```

Screenshot Hasil:



Gambar F.3 – Screenshot Hasil Proyek Mini

G. PEMBAHASAN

G.1 Analisis Kesesuaian Hasil dengan Teori

Hasil praktikum menunjukkan kesesuaian yang baik dengan teori Asynchronous JavaScript yang diajarkan. Penggunaan fetch() berhasil mengambil data dari API eksternal secara non-blocking, dibuktikan dengan indikator loading yang tetap berjalan dan UI tidak membeku saat menunggu respons server. Method .then() dan .catch() bekerja sesuai teori **Promise** untuk menangani success dan error state. Data JSON yang diterima berhasil di-parse dan dirender menjadi elemen HTML secara dinamis, membuktikan pemahaman tentang DOM Manipulation dan JSON (JavaScript Object Notation). Fungsi tindakLanjut() yang dipanggil melalui onclick menunjukkan implementasi **event** handling yang tepat. Secara keseluruhan, output yang dihasilkan sesuai dengan ekspektasi teori asynchronous programming.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Data dari JSONPlaceholder /posts menggunakan teks Lorem Ipsum (bahasa Latin) sehingga judul dan deskripsi insiden tidak relevan dengan konteks industri manufaktur.	Memahami bahwa JSONPlaceholder adalah API simulasi untuk pembelajaran. Fokus diarahkan pada **fungsionalitas kode**, bukan pada konten data. Jika diperlukan data yang lebih relevan, dapat mengganti dengan array data buatan sendiri.
Deskripsi insiden (body) terlalu panjang sehingga card menjadi tidak rapi dan memakan banyak ruang.	Menggunakan method .substring(0, 100) + '...' untuk membatasi tampilan deskripsi menjadi 100 karakter pertama, menjaga kerapian layout card.
Karakter petik tunggal (') dalam judul insiden dapat merusak sintaks onclick di HTML.	Menggunakan .replace(/'/g, "\\'") untuk **escape** karakter petik tunggal sebelum dimasukkan ke dalam atribut onclick.

G.3 Kaitan dengan Penerapan di Industri

Proyek mini ini merepresentasikan sistem **pelaporan dan penanganan insiden** yang umum digunakan di lantai produksi. Dalam industri manufaktur, aplikasi serupa dapat dikembangkan sebagai dashboard maintenance yang terhubung dengan API server pabrik untuk menampilkan tiket kerusakan mesin secara real-time. Operator atau supervisor dapat melihat daftar insiden terbaru dan menekan tombol "Tindak Lanjut" sebagai konfirmasi bahwa masalah sedang ditangani. Konsep asynchronous fetching yang dipelajari menjadi dasar penting untuk mengintegrasikan aplikasi frontend dengan sistem backend seperti CMMS (Computerized Maintenance Management System) atau MES (Manufacturing Execution System) di era Industri 4.0.

H. KESIMPULAN

1	Mahasiswa berhasil mengimplementasikan Fetch API untuk ambil data dari endpoint publik JSONPlaceholder dan menampilkannya dalam bentuk card Bootstrap yang responsif.
2	Mahasiswa mampu menerapkan konsep Asynchronous programming dengan benar, ditunjukkan oleh indikator loading yang muncul selama proses fetch dan UI yang tetap interaktif.
3	Mahasiswa memahami cara membatasi dan memformat data menggunakan method JavaScript seperti .slice( ) untuk mengambil 10 data pertama dan .substring( ) untuk memotong teks deskripsi.
4	Mahasiswa berhasil menambahkan interaktivitas melalui tombol “Tindak Lanjut” yang memicu alert, mensimulasikan alur kerja penanganan insiden di lingkungan industri.
5	Mahasiswa mampu mengidentifikasi dan mengatasi kendala teknis seperti escape karakter dan global scope function, menunjukkan kemampuan problem-solving dalam pengembangan web.

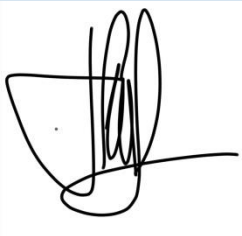
I. DAFTAR PUSTAKA

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	Dasmito, M. W. (2019). *Analisis Performa Aplikasi Web Service Worker dengan Metode Pengujian YSlow* [Skripsi, Universitas Hasanuddin]. Repository Universitas Hasanuddin. http://repository.unhas.ac.id/id/eprint/3139/
[3]	Ndunagu, J. N., Ekwueme, C. O., & Onah, C. A. (2021). Web-based application developmer using HTML5 Web Storage for offline data management. *International Journal of Advanced Computer Science and Applications (IJACSA)*, 12(3), 210–217. https://doi.org/10.14569/IJACSA.2021.0120325
[4]	Severance, C. (2012). JavaScript: Designing a language in 10 days. *IEEE Computer*, 45(2), https://doi.org/10.1109/MC.2012.57

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
	TOTAL NILAI AKHIR	100%			

Mahasiswa	Diperiksa oleh Asisten / Dosen
	
Vanyra Raraswara Eka Putri NIM: 23051430013	Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT. Tanggal: _____